A Distributed Hadoop-Spark Framework for Predictive Maintenance and Remaining Useful Life Prediction

Shiwani Banjare
Dept. of DSAI, IIIT Naya Raipur
Raipur, Chhattisgarh, India
shiwani23102@iiitnr.edu.in

Khushboo Painkra
Dept. of DSAI, IIIT Naya Raipur
Raipur, Chhattisgarh, India
khushboo23102@iiitnr.edu.in

Sakshi Sonker
Dept. of CSE, IIIT Naya Raipur
Raipur, Chhattisgarh, India
sakshi23100@iiitnr.edu.in

Prof. Mallikarjuna Rao
Dept. of DSAI, IIIT Naya Raipur
Raipur, Chhattisgarh, India
mallikarjuna@iiitnr.edu.in

***Abstract*—Modern industrial systems generate massive vol umes of sensor data that require scalable storage, distributed processing, and efficient predictive analytics for real-time main tenance applications. Traditional single-node machine learning systems often face limitations in handling large-scale industrial telemetry data due to scalability, computational overhead, and storage constraints. This paper presents a distributed predictive maintenance framework using the Hadoop and Spark ecosystem for Remaining Useful Life prediction and engine failure clas sification. A multi-node cluster architecture was implemented using one master node and multiple slave nodes interconnected through Secure Shell communication and distributed resource management. Hadoop Distributed File System was used for fault tolerant distributed storage, while Apache Spark enabled parallel data preprocessing, feature engineering, and distributed machine learning. Hive was integrated for SQL-based distributed analyt ics, and MapReduce was utilized for parallel sensor aggregation operations. Multiple machine learning models including Linear Regression, Random Forest, Gradient Boosted Trees, Logistic Regression, and Mahout-based distributed Logistic Regression were implemented and evaluated on the NASA Turbofan Engine Degradation dataset. Feature engineering techniques such as Remaining Useful Life generation, rolling sensor averages, and Remaining Useful Life capping significantly improved predic tion performance. Experimental results demonstrated that the optimized Gradient Boosted Tree model achieved a Root Mean Square Error of 18.19, while the Logistic Regression classification model achieved an accuracy and F1 score of 94.69 percent. The proposed framework demonstrates the effectiveness of combin ing distributed Big Data infrastructure with scalable machine learning for industrial predictive maintenance applications.**

***Index Terms*—Big Data, Predictive Maintenance, Hadoop, Apache Spark, HDFS, Distributed Computing, Machine Learning, Remaining Useful Life, Hive, MapReduce, Mahout**

I. Introduction

The rapid growth of Industry 4.0 and Industrial Internet of Things technologies has led to the generation of massive volumes of industrial sensor data from machines, engines, and manufacturing systems. Modern industrial equipment continuously produces operational and telemetry data that can be analyzed to monitor system health and detect possible failures before they occur.

Predictive maintenance has emerged as an important approach for reducing unexpected machine breakdowns, minimizing maintenance costs, and improving operational efficiency [4]. Unlike traditional reactive or scheduled maintenance approaches, predictive maintenance utilizes historical and real-time sensor data to estimate machine degradation and predict future failures.

However, handling large-scale industrial datasets using traditional single-node systems presents several challenges, including limited scalability, high computational overhead, slow processing speed, and storage inefficiency. As industrial environments generate continuously increasing volumes of sensor information, distributed Big Data frameworks have become essential for scalable data storage, processing, and machine learning applications.

Technologies such as Hadoop Distributed File System, Apache Spark, Hive, and Mahout provide distributed and fault-tolerant architectures capable of handling large-scale industrial analytics efficiently [1], [2], [7], [8].

This paper presents a distributed predictive maintenance framework using the Hadoop and Spark ecosystem for Remaining Useful Life prediction and engine failure classification. A multi-node cluster architecture consisting of one master node and multiple slave nodes was implemented using distributed storage and parallel processing techniques.

Hadoop Distributed File System was used for reliable distributed storage with block replication and fault tolerance, while Apache Spark enabled parallel preprocessing, feature engineering, and machine learning model training. Hive was integrated for SQL-based distributed analytics, and MapReduce was utilized for distributed sensor aggregation operations.

Multiple machine learning models including Linear Re-

gression, Random Forest, Gradient Boosted Trees, Logistic Regression, and Mahout-based distributed Logistic Regression were implemented and evaluated using the NASA Turbofan Engine Degradation dataset.

The proposed framework focuses not only on predictive accuracy but also on distributed system design, scalability, cluster communication, and efficient Big Data processing.

Experimental results demonstrate that the optimized Gradient Boosted Tree model achieved a Root Mean Square Error of 18.19, while the Logistic Regression classification model achieved an accuracy and F1 score of 94.69%.

The overall system demonstrates the effectiveness of integrating distributed Big Data infrastructure with scalable machine learning for industrial predictive maintenance applications.

The main contributions of this work are as follows:

- Development of a distributed Hadoop-Spark framework for predictive maintenance.
- Integration of HDFS, Spark MLlib, Hive, MapReduce, and Mahout within a unified architecture.
- Implementation of scalable Remaining Useful Life prediction and failure classification models.
- Performance improvement using feature engineering and distributed processing techniques.

The remainder of this paper is organized as follows. Section II presents the literature review and related work. Section III describes the proposed distributed system architecture and methodology. Section IV discusses the implementation details of the Hadoop and Spark cluster environment. Section V presents experimental results and performance evaluation. Finally, Section VI concludes the paper and discusses future research directions.

II. Literature Review

Predictive maintenance has become an important research area in industrial systems due to the increasing availability of sensor-generated operational data. Traditional maintenance strategies such as reactive maintenance and preventive maintenance often lead to increased operational costs, unnecessary component replacement, and unexpected machine downtime.

To overcome these limitations, researchers have increasingly focused on machine learning and Big Data techniques for failure prediction and Remaining Useful Life (RUL) estimation.

Several studies have applied machine learning algorithms for predictive maintenance using industrial sensor datasets. Regression-based approaches such as Linear Regression and Support Vector Regression (SVR) have been widely used for RUL prediction because of their simplicity and interpretability. However, these approaches often fail to capture nonlinear degradation patterns present in complex industrial systems.

Ensemble learning methods such as Random Forest and Gradient Boosted Trees have shown improved performance due to their ability to model nonlinear relationships and handle high-dimensional sensor data efficiently [9], [10].

Deep learning approaches including Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks have also been explored for predictive maintenance applications. These models are capable of learning temporal degradation behavior from sequential sensor readings. Although deep learning methods can achieve high prediction accuracy, they generally require high computational resources, large training datasets, and complex hardware infrastructure.

Many existing approaches primarily focus on model accuracy while giving limited attention to distributed scalability, fault tolerance, and large-scale data processing.

With the rapid increase in industrial telemetry data, Big Data technologies have become essential for scalable predictive maintenance systems.

Hadoop Distributed File System (HDFS) has been widely adopted for distributed storage of large datasets due to its fault tolerance and block replication capabilities [1].

Apache Spark has gained popularity for distributed data processing and machine learning because of its in-memory computation and parallel execution model [2], [5].

Hive provides SQL-based distributed analytics over Hadoop clusters, while MapReduce enables parallel batch processing of massive datasets [6], [7].

Several researchers have proposed cloud-based and distributed predictive maintenance systems using Hadoop and Spark frameworks. However, many existing systems either focus only on distributed storage or only on machine learning implementation.

Limited work has been conducted on integrating distributed cluster communication, HDFS storage, Spark-based machine learning, Hive analytics, MapReduce operations, and Mahout-based distributed learning within a single unified framework.

The proposed work addresses these limitations by developing a complete distributed predictive maintenance architecture using the Hadoop ecosystem.

Unlike traditional single-node machine learning approaches, the proposed framework combines distributed storage, parallel processing, cluster-based machine learning, and fault-tolerant system design.

The framework integrates HDFS for distributed storage, Spark MLlib for scalable machine learning, Hive for distributed SQL analytics, MapReduce for sensor aggregation, and Mahout for distributed Logistic Regression.

Furthermore, feature engineering techniques such as Remaining Useful Life generation, rolling sensor averages, and RUL capping are incorporated to improve prediction performance.

Table I presents a comparison between existing predictive maintenance approaches and the proposed distributed framework.

The literature review demonstrates that although significant progress has been made in predictive maintenance research, there remains a need for scalable and fully integrated Big Data frameworks capable of handling large-scale industrial telemetry data efficiently.

The proposed work contributes toward addressing this research gap through a distributed and fault-tolerant predictive maintenance architecture.

TABLE I: Comparison of Existing Predictive Maintenance Approaches

Existing Work	Technique Used	Limitation
Lee et al. [4]	Industry 4.0 Predictive Systems	Limited distributed ML support
Malhotra et al.	LSTM-based Predictive Maintenance	High computational complexity
Zaharia et al. [2]	Spark Distributed Processing	Focused mainly on processing
Dean and Ghemawat [7]	MapReduce Batch Processing	No integrated ML pipeline
Proposed Framework	Hadoop, Spark, Hive, MapReduce, Mahout	Scalable integrated architecture

III. Proposed System Architecture and Methodology

The proposed system presents a distributed predictive maintenance framework for industrial equipment failure prediction using the Hadoop and Spark Big Data ecosystem.

The framework integrates distributed storage, parallel data processing, distributed analytics, and machine learning techniques to perform scalable Remaining Useful Life (RUL) prediction and engine failure classification using industrial sensor telemetry data.

The system was designed to address the challenges associated with large-scale industrial data processing, including high data volume, computational overhead, fault tolerance, and real-time analytical scalability.

To achieve efficient distributed computation, the framework combines Hadoop Distributed File System (HDFS), Apache Spark, Apache Hive, MapReduce, and Apache Mahout within a multi-node cluster architecture [1], [2], [6]–[8].

The proposed framework enables distributed storage, parallel preprocessing, feature engineering, machine learning model training, and predictive analytics across multiple worker nodes, thereby reducing computational latency and improving scalability for industrial predictive maintenance applications.

A. Overall System Architecture

The proposed architecture consists of a distributed Hadoop-Spark cluster composed of one master node and multiple slave nodes connected through Secure Shell (SSH)-based communication.

The cluster was configured on Ubuntu Linux systems using static IP addressing and hostname mapping for reliable distributed communication.

The master node was configured to manage cluster coordination, metadata management, and distributed resource allocation.

The following services were deployed on the master node:

- HDFS NameNode
- YARN ResourceManager
- Spark Master
- Hive Metastore and Hive Services

The slave nodes were configured to perform distributed storage and parallel computation.

Each slave node hosted:

- DataNode
- YARN NodeManager
- Spark Worker

Passwordless SSH authentication was configured between cluster nodes to enable seamless distributed task execution.

Hadoop and Spark configuration files including:

- core-site.xml
- hdfs-site.xml
- mapred-site.xml
- yarn-site.xml
- workers

were modified to establish distributed communication and cluster management.

The overall workflow of the framework is illustrated through the following stages:

The distributed architecture enables parallel execution of computational tasks across multiple worker nodes, thereby improving processing efficiency and system scalability.

B. Distributed Storage Using HDFS

The NASA Turbofan Engine Degradation Dataset was used as the primary industrial dataset for predictive maintenance analysis.

The dataset contains multiple engine operational cycles along with sensor telemetry readings and operational settings collected during engine degradation.

The dataset was uploaded into Hadoop Distributed File System (HDFS), which provides scalable and fault-tolerant distributed storage capabilities [1].

HDFS divides large datasets into smaller fixed-size blocks and distributes them across multiple DataNodes within the cluster.

The NameNode maintains metadata information including:

- File locations
- Block mappings
- Replication details

while DataNodes physically store the actual dataset blocks.

To improve reliability and fault tolerance, block replication was enabled across multiple slave nodes [1].

In the event of node failure, replicated blocks remain accessible from other DataNodes, ensuring uninterrupted data availability and cluster reliability.

The distributed storage architecture enables Spark, Hive, and MapReduce components to access the dataset concurrently for parallel processing and analytics.

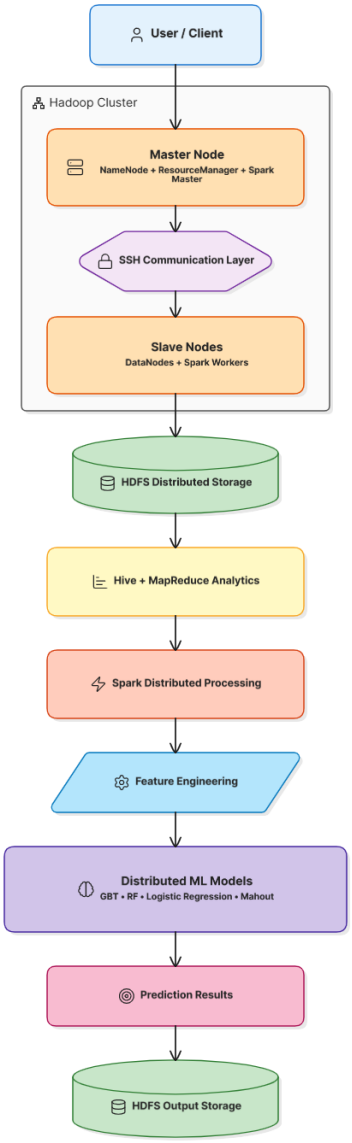

Fig. 1: Proposed Distributed Predictive Maintenance Architecture

C. Data Preprocessing and Feature Engineering

The preprocessing stage was performed using Apache Spark DataFrames to enable distributed in-memory computation across worker nodes.

Since industrial telemetry datasets often contain noisy and incomplete sensor measurements, multiple preprocessing operations were performed before machine learning model training.

1) Data Cleaning: The following preprocessing operations were implemented:

- Removal of null or empty columns
- Assignment of meaningful feature names
- Conversion of raw sensor data into structured DataFrames
- Elimination of redundant operational attributes

These operations improved dataset consistency and model interpretability.

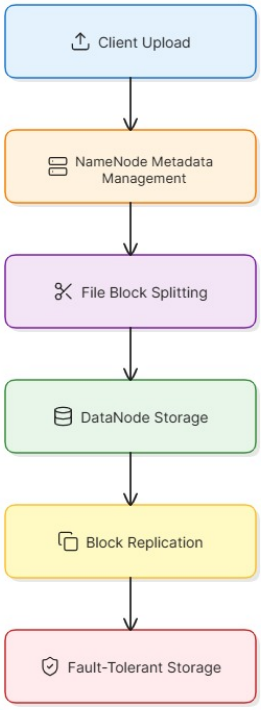

Fig. 2: HDFS Distributed Storage Workflow

2) Remaining Useful Life Generation: Remaining Useful Life (RUL) values were generated by calculating the difference between the maximum operational cycle of an engine and its current operational cycle.

The RUL generation equation is given as

$$RUL = \max(Cycle) - CurrentCycle \tag{1}$$

where RUL represents the Remaining Useful Life of the engine, $\max(Cycle)$ denotes the maximum operational cycle of a particular engine, and $CurrentCycle$ represents the current cycle number.

The generated RUL values were used as supervised learning labels for predictive maintenance modeling.

3) RUL Capping: To reduce prediction instability caused by extremely large degradation cycles, RUL capping was applied using a threshold value of 125 cycles.

This technique reduces variance in target labels and improves model generalization performance.

4) Rolling Sensor Feature Engineering: Rolling sensor averages were computed using Spark window operations to capture temporal degradation behavior of industrial engines.

Feature engineering included:

- Operational settings
- Sensor telemetry values
- Rolling sensor averages
- Aggregated statistical features

These engineered features improved the capability of machine learning models to identify degradation trends and nonlinear failure patterns.

D. Hive-Based Distributed Analytics

Apache Hive was integrated into the framework to support SQL-based distributed querying over data stored in HDFS [6].

Hive external tables were created directly over the distributed dataset without duplicating data.

Hive enabled structured analytical operations including:

- Aggregation queries
- Filtering operations
- Grouping operations
- Statistical analysis
- Sensor trend analysis

Hive queries were internally translated into distributed execution jobs, enabling scalable analytical processing over large industrial datasets.

The integration of Hive simplified interaction with distributed data using SQL-like syntax while maintaining Hadoop scalability advantages.

E. MapReduce-Based Distributed Processing

MapReduce was implemented to demonstrate distributed batch processing and sensor aggregation over industrial telemetry data [7].

Custom Mapper and Reducer scripts were developed using Python.

1) Mapper Phase: The Mapper extracted engine identifiers and sensor values from the dataset and generated intermediate key-value pairs.

For example:

- Key $\rightarrow$ Engine ID
- Value $\rightarrow$ Sensor Reading

2) Shuffle and Sort Phase: The Hadoop framework automatically grouped intermediate outputs based on identical keys and distributed them across reducers.

3) Reducer Phase: The Reducer computed aggregated sensor statistics including:

- Average sensor values
- Sensor-wise operational summaries
- Engine-level aggregated measurements

The MapReduce implementation demonstrated scalable distributed computation over large-scale industrial telemetry datasets.

F. Distributed Machine Learning Using Spark MLlib

Apache Spark MLlib was used to implement distributed machine learning algorithms for predictive maintenance prediction and failure analysis [2], [5].

Spark partitions the dataset across worker nodes and performs parallel training and inference operations, significantly reducing computational overhead compared to centralized machine learning systems.

Several machine learning models were implemented and evaluated.

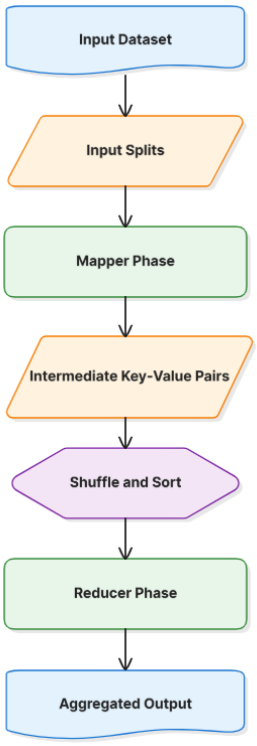

Fig. 3: MapReduce Sensor Aggregation Workflow

1) Linear Regression: Linear Regression was implemented for baseline Remaining Useful Life prediction by modeling linear relationships between sensor features and degradation cycles.

The Linear Regression model is represented as

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \epsilon \tag{2}$$

where:

- y represents the predicted Remaining Useful Life
- β_0 is the intercept term
- $\beta_1, \beta_2, \ldots, \beta_n$ represent regression coefficients
- $x_1, x_2, \ldots, x_n$ denote sensor input features
- ϵ represents the error term

2) Random Forest Regressor: Random Forest Regressor was implemented to improve prediction performance using ensemble learning techniques.

Multiple decision trees were trained in parallel, and final predictions were generated using aggregated outputs from all trees.

The algorithm improved robustness against overfitting and handled nonlinear sensor relationships effectively.

3) Gradient Boosted Tree Regressor: Gradient Boosted Trees (GBT) were implemented for advanced nonlinear degradation modeling.

GBT sequentially trains decision trees where each new tree minimizes prediction errors from previous trees.

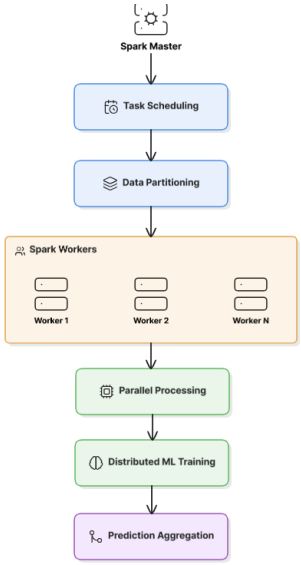

Fig. 4: Spark Distributed Processing Workflow

After hyperparameter tuning and feature engineering, Gradient Boosted Trees achieved the best predictive performance among all implemented models.

4) Logistic Regression: Logistic Regression was implemented for binary engine failure classification by converting Remaining Useful Life values into failure and non-failure labels.

The Logistic Regression model is represented as

$$P(Y=1) = \frac{1}{1+e^{-z}} \tag{3}$$

where $P(Y=1)$ represents the probability of engine failure and z represents the weighted linear combination of input features.

This classification model enabled prediction of imminent engine failure conditions.

G. Mahout-Based Distributed Classification

Apache Mahout was integrated into the framework to demonstrate Hadoop-based distributed machine learning capabilities [8].

Mahout-based Logistic Regression was trained using engineered sensor features stored within HDFS.

The implementation demonstrated interoperability between:

- HDFS distributed storage
- Hadoop ecosystem components
- Distributed classification algorithms

Mahout extended the framework's capability for scalable distributed classification within the Hadoop ecosystem.

H. Model Evaluation Metrics

The performance of predictive maintenance models was evaluated using Root Mean Square Error (RMSE), which measures prediction deviation between actual and predicted Remaining Useful Life values.

The RMSE equation is given as

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{4}$$

where:

- y_i represents actual Remaining Useful Life values
- $\hat{y}_i$ represents predicted Remaining Useful Life values
- n denotes the total number of observations

Lower RMSE values indicate better predictive accuracy and improved Remaining Useful Life estimation.

Additional comparative analysis was performed across all implemented machine learning algorithms to identify the most effective predictive maintenance model.

IV. Experimental Setup

The experimental evaluation of the proposed predictive maintenance framework was conducted using a distributed Hadoop-Spark cluster environment.

The setup was designed to evaluate distributed storage, preprocessing efficiency, machine learning performance, and scalability for large-scale industrial sensor analytics.

The implementation integrated Hadoop Distributed File System (HDFS), Apache Spark, Hive, MapReduce, and Mahout within a multi-node cluster consisting of one master node and three slave nodes.

Distributed processing enabled parallel execution of preprocessing and machine learning operations over industrial telemetry data.

A. Dataset

The experiments were performed using the NASA CMAPSS Turbofan Engine Degradation Simulation Dataset [3].

The dataset contains simulated industrial engine degradation data collected under multiple operational conditions and failure scenarios.

The dataset includes:

- Engine operational settings
- Sensor measurements
- Engine operational cycles
- Failure degradation information

The dataset contains 21 sensor values, 3 operational settings, and engine cycle information.

The predictive target used in the proposed framework was Remaining Useful Life (RUL).

The dataset was uploaded into HDFS using distributed file operations and accessed by Spark, Hive, and MapReduce components during preprocessing and analytics.

B. Cluster and System Configuration

The framework was implemented using Ubuntu Linux virtual machines configured in a distributed multi-node cluster environment.

The cluster consisted of:

- One master node

- Three slave nodes

Each node was assigned a static IP address to establish stable communication within the cluster.

Hostname mappings were configured using Linux host configuration files.

Secure Shell (SSH)-based passwordless authentication was configured between the master and slave nodes to enable automatic distributed communication during Hadoop and Spark execution.

The master node hosted:

- HDFS NameNode
- YARN ResourceManager
- Spark Master
- Hive Services

The slave nodes hosted:

- HDFS DataNodes
- YARN NodeManagers
- Spark Worker Nodes

The Hadoop workers configuration file and Spark workers configuration file were updated to register all worker nodes within the cluster.

C. Software Configuration

The proposed framework was implemented using the following software components.

TABLE II: Software Configuration

Software / Framework	**Purpose**
Ubuntu Linux	Operating System
Hadoop HDFS	Distributed Storage
YARN	Resource Management
Apache Spark	Distributed Processing
Spark MLlib	Machine Learning
Apache Hive	SQL Analytics
Apache Mahout	Distributed Classification
Hadoop MapReduce	Batch Processing
Python	Development Language

D. HDFS and Spark Setup

HDFS was configured to provide distributed storage for industrial sensor datasets.

The NameNode managed metadata information and block allocation, while DataNodes stored dataset blocks across worker nodes.

The replication mechanism of HDFS ensured fault tolerance by maintaining replicated copies of data blocks across multiple slave nodes [1].

Apache Spark was configured in standalone cluster mode using one Spark Master and multiple Spark Worker nodes.

PySpark was used for distributed preprocessing, feature engineering, and machine learning operations over datasets stored in HDFS.

E. Experimental Parameters

Several preprocessing and machine learning parameters were configured during experimentation to improve predictive maintenance performance.

TABLE III: Preprocessing Parameters

Parameter	**Value / Description**
Null Column Handling	Removed
Feature Engineering	Rolling Sensor Averages
RUL Threshold	125
Replication Factor	3

Remaining Useful Life values were generated using

$$RUL = \max(Cycle) - Cycle \tag{5}$$

RUL capping was applied using

$$RUL_{capped} = \min(RUL, 125) \tag{6}$$

This preprocessing step reduced prediction instability caused by extremely large Remaining Useful Life values and improved model generalization performance.

F. Machine Learning Setup

Distributed machine learning models were implemented using Spark MLlib and Mahout.

Spark distributed the dataset across worker nodes and executed preprocessing and training operations in parallel.

The following machine learning models were trained and evaluated:

- Linear Regression
- Random Forest Regressor
- Gradient Boosted Tree Regressor
- Logistic Regression
- Mahout Logistic Regression

Feature engineering techniques including rolling sensor averages, Remaining Useful Life generation, and RUL capping were incorporated during training to improve predictive performance.

G. Evaluation Metrics

The predictive maintenance models were evaluated using the following performance metrics:

- Root Mean Square Error (RMSE)
- Accuracy
- F1 Score

These metrics were used to compare predictive performance and classification effectiveness across different distributed machine learning models.

V. Experimental Results and Discussion

The proposed distributed predictive maintenance framework was evaluated using the NASA Turbofan Engine Degradation dataset.

Experiments were conducted to analyze the performance of distributed machine learning models for Remaining Useful Life (RUL) prediction and engine failure classification.

A. Experimental Environment

The experiments were performed on a multi-node Hadoop and Spark cluster consisting of one master node and three slave nodes [1], [2].

HDFS was used for distributed storage, while Spark MLlib and Mahout were used for distributed machine learning [1], [2], [8].

The dataset consisted of operational settings and sensor measurements collected from multiple turbofan engines operating under different degradation conditions.

B. Remaining Useful Life Prediction Results

Multiple regression models were evaluated for Remaining Useful Life prediction.

TABLE IV: RUL Prediction Performance

Model	**RMSE**
Linear Regression	44.85
Random Forest Regressor	41.50
Initial GBT Regressor	18.95
Optimized GBT Regressor	18.19

The optimized Gradient Boosted Tree model achieved the best performance with the lowest Root Mean Square Error value of 18.19.

The improvement was achieved through feature engineering, rolling sensor averages, Remaining Useful Life capping, and hyperparameter tuning.

The results demonstrate that ensemble learning techniques are more effective in capturing nonlinear degradation behavior present in industrial telemetry data [9], [10].

C. Failure Classification Results

Logistic Regression was implemented for binary failure classification using failure labels generated from Remaining Useful Life values.

TABLE V: Failure Classification Performance

Metric	**Value**
Accuracy	94.69%
F1 Score	94.69%

The confusion matrix generated from classification predictions is shown in Table IX.

The results indicate that the proposed framework successfully identified engine failure conditions with high classification performance.

TABLE VI: Confusion Matrix for Failure Classification

Actual Label	**Predicted Label**	**Count**
No Failure	No Failure	3336
Failure	Failure	502
Failure	No Failure	107
No Failure	Failure	108

D. Mahout Logistic Regression Results

Mahout-based Logistic Regression was implemented to demonstrate distributed machine learning capabilities within the Hadoop ecosystem [8].

Mahout successfully trained classification models using distributed sensor features and generated feature importance coefficients.

Sensor features including sensor2, sensor3, sensor4, sensor7, and rolling sensor averages contributed significantly toward engine failure classification.

E. Discussion

The experimental results demonstrate the effectiveness of combining distributed Big Data infrastructure with machine learning for predictive maintenance applications.

The use of HDFS enabled fault-tolerant distributed storage, while Spark improved processing speed through parallel execution [1], [2].

Hive simplified analytical querying over distributed datasets, and MapReduce demonstrated scalable batch processing capabilities [6], [7].

Feature engineering played a critical role in improving predictive performance. Remaining Useful Life capping reduced prediction imbalance caused by extremely large cycle values, while rolling sensor averages captured degradation trends more effectively.

The proposed distributed architecture also improved scalability and system reliability compared to traditional single-node predictive maintenance systems.

Experimental results demonstrated that the optimized Gradient Boosted Tree model achieved a Root Mean Square Error of 18.19, while the Logistic Regression classification model achieved an accuracy and F1 score of 94.69%.

The proposed work highlights the effectiveness of integrating distributed Big Data infrastructure with scalable machine learning for industrial predictive maintenance applications [1], [2], [5].

The proposed framework demonstrates that distributed Big Data ecosystems can significantly improve predictive maintenance scalability, computational efficiency, and industrial decision-making capabilities in modern Industry 4.0 environments.

The framework provides a strong foundation for future large-scale industrial analytics systems capable of handling continuously increasing volumes of sensor data efficiently.

VI. Future Work

Several improvements can be incorporated into the proposed framework in future work.

- Integration of Apache Kafka for real-time sensor streaming
- Deployment on cloud-based distributed environments
- Kubernetes-based cluster orchestration
- Integration with Industrial Internet of Things devices of deep learning models such as LSTM and Transformer networks for advanced predictive maintenance analytics [21], [22]
- Real-time dashboard visualization for industrial monitoring
- Edge computing integration for low-latency predictive analytics

Future work can further improve scalability, real-time analytics, and intelligent maintenance decision-making capabilities.

VII. Limitations

The proposed framework was evaluated using simulated industrial telemetry data from the NASA CMAPSS dataset.

Real-world industrial deployments may involve additional challenges including sensor noise variability, hardware heterogeneity, real-time streaming constraints, and large-scale deployment costs.

Furthermore, deep learning-based temporal architectures were not evaluated due to computational resource limitations.

Although the proposed framework demonstrated strong predictive performance, additional validation using real industrial production environments would further strengthen system reliability and scalability analysis.

VIII. Conclusion

This paper presented a distributed predictive maintenance framework using the Hadoop and Spark ecosystem for Remaining Useful Life prediction and engine failure classification [1], [2].

A complete multi-node distributed architecture consisting of one master node and multiple slave nodes was implemented using Hadoop Distributed File System, Spark, Hive, MapReduce, and Mahout [1], [2], [6]–[8].

The proposed framework demonstrated scalable storage, distributed processing, fault tolerance, and parallel machine learning capabilities for industrial telemetry analytics.

Feature engineering techniques including Remaining Useful Life generation, rolling sensor averages, and RUL capping significantly improved predictive performance.

Experimental results demonstrated that the optimized Gradient Boosted Tree model achieved the best predictive performance with an RMSE of 18.19, while Logistic Regression achieved 94.69% classification accuracy.

The proposed work highlights the effectiveness of integrating distributed Big Data infrastructure with scalable machine learning for industrial predictive maintenance applications [1], [2], [5].

References

[1] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop Distributed File System," *IEEE 26th Symposium on Mass Storage Systems and Technologies*, pp. 1–10, 2010.

[2] M. Zaharia et al., "Apache Spark: A Unified Engine for Big Data Processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[3] A. Saxena and K. Goebel, "Turbofan Engine Degradation Simulation Data Set," NASA Ames Prognostics Data Repository, 2008.

[4] J. Lee, B. Bagheri, and H. A. Kao, "A Cyber-Physical Systems architecture for Industry 4.0-based manufacturing systems," *Manufacturing Letters*, vol. 3, pp. 18–23, 2014.

[5] X. Meng et al., "MLlib: Machine Learning in Apache Spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.

[6] A. Thusoo et al., "Hive – A Warehousing Solution Over a Map-Reduce Framework," *Proceedings of the VLDB Endowment*, vol. 2, no. 2, pp. 1626–1629, 2009.

[7] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.

[8] S. Owen, R. Anil, T. Dunning, and E. Friedman, "Mahout in Action," Manning Publications, 2011.

[9] J. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[10] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[11] P. Malhotra et al., "LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection," ICML Workshop, 2016.

[12] T. White, *Hadoop: The Definitive Guide*, O'Reilly Media, 2015.

[13] A. Verma et al., "Large-scale cluster management at Google with Borg," *European Conference on Computer Systems*, 2015.

[14] F. Chollet, *Deep Learning with Python*, Manning Publications, 2018.

[15] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.

[16] Apache Hadoop Documentation, Apache Software Foundation.

[17] Apache Spark Documentation, Apache Software Foundation.

[18] Apache Hive Documentation, Apache Software Foundation.

[19] Apache Mahout Documentation, Apache Software Foundation.

[20] Ubuntu Linux Documentation, Canonical Ltd.

[21] Y. Zhang et al., "Deep Learning-Based Predictive Maintenance for Industrial IoT," IEEE Access, vol. 9, pp. 821–832, 2021.

[22] S. Wang et al., "Remaining Useful Life Prediction Using LSTM Networks," Reliability Engineering and System Safety, vol. 210, 2021.

[23] A. Carvalho et al., "A Systematic Literature Review of Machine Learning Methods Applied to Predictive Maintenance," Computers and Industrial Engineering, vol. 137, 2019.

[24] M. Canizo et al., "Real-time Predictive Maintenance for Wind Turbines Using Spark Streaming," IEEE Transactions on Sustainable Computing, vol. 5, no. 2, 2020.

[25] H. Zhang et al., "Industrial Big Data Analytics for Predictive Maintenance Applications," Journal of Manufacturing Systems, vol. 54, pp. 128–137, 2020.